

Билет №1: Алгоритм: определение и свойства. Анализ сложности: память и время как ресурсы.

1. Алгоритм: определение и свойства

Определение

Алгоритм — это набор конечного числа правил, задающих последовательность выполнения операций компьютерной программой для решения задачи определённого типа.

Основные свойства (характеристики) алгоритма:

- **Дискретность.** Алгоритм должен состоять из конкретного набора операций, следующих друг за другом. Это означает, что процесс решения задачи разбит на отдельные шаги, и выполнение каждого шага строго определено.
- **Конечность.** Алгоритм всегда должен заканчиваться после выполнения конечного числа шагов. Для корректного алгоритма недопустимо заикливание или бесконечное выполнение; он обязан привести к результату (успешному или неуспешному) за конечное время.
- **Определённость (детерминированность).** Действия, которые нужно выполнить, должны быть строго и недвусмысленно определены для каждого возможного случая. Это значит, что для одного и того же набора входных данных алгоритм должен всегда выдавать один и тот же результат и проходить через одну и ту же последовательность шагов. В нем нет места для случайности или неоднозначности.
- **Массовость.** Алгоритм должен решать целый класс однотипных задач, а не одну конкретную проблему. Например, алгоритм сортировки должен уметь сортировать *любой* массив чисел, а не только заранее заданный $[3, 1, 2]$.

2. Анализ сложности: память и время как ресурсы

• Анализ пространственной сложности (память):

- Используется для оценки необходимой памяти как функции от размера входных данных.
- Согласно слайду 18, существует четыре аспекта использования памяти:
 1. Память для хранения кода самого алгоритма.
 2. Память для входных данных.
 3. Память для выходных данных.
 4. Память, необходимая для вычислительного процесса во время вычислений (промежуточные данные, стек вызовов и т.д.).

• Анализ временной сложности (время):

- Используется для оценки времени работы алгоритма как функции от размера входных данных. Этот анализ позволяет понять, как быстро растёт время выполнения при увеличении объема обрабатываемой информации.



Билет №2: O-символика как инструмент оценки ресурсов. Основные типы асимптотик.

1. O-символика («O» большое)

- **Происхождение:** Термин происходит от немецкого слова «Ordnung» (порядок) и был введен Паулем Бахманом в 1894 году.
- **Определение:** «O» большое — это математическое обозначение для сравнения асимптотического поведения (асимптотики) функций. Под асимптотикой понимается характер изменения функции, когда её аргумент стремится к бесконечности (или к определенной точке).
- **Строгое математическое определение:** Запись $O(f(n))$ означает, что существуют положительные константы M и n_0 , такие, что величина $|x_n|$ (например, время работы алгоритма), представленная как $O(f(n))$, удовлетворяет условию $|x_n| \leq M|f(n)|$ для всех целых $n \geq n_0$. Проще говоря, начиная с некоторого размера задачи (n_0), время работы не будет превышать функцию $f(n)$, умноженную на константу M .

2. O-символика как инструмент оценки ресурсов

Фраза «**сложность алгоритма есть ($O(f(n))$)**» означает, что с увеличением параметра (n) (характеризующего размер входных данных), время работы алгоритма (или требуемая память) будет возрастать **не быстрее**, чем некоторая константа, умноженная на ($f(n)$). Это позволяет игнорировать машинные особенности (разную скорость процессора) и мелкие детали реализации, фокусируясь на основной тенденции роста затрат ресурсов.

3. Основные типы асимптотик

Хотя в первой лекции напрямую не перечисляются все типы, на слайде 24 приведены свойства O -символики, а на слайдах 25-36 на примере бинарного поиска демонстрируются основные классы. Самым важным примером является логарифмическая сложность:

- **Логарифмическая сложность ($O(\log n)$)**: Демонстрируется на примере бинарного поиска.
 - На слайдах 29-36 показано, как количество шагов (сравнений) алгоритма растёт при увеличении размера массива (n):
 - ($n = 1$) $\rightarrow$ 1 шаг
 - ($n = 4$) $\rightarrow$ 3 шага
 - ($n = 8$) $\rightarrow$ 4 шага
 - ($n = 16$) $\rightarrow$ 5 шагов
 - ($n = 32$) $\rightarrow$ 6 шагов
 - На слайдах 37-38 выводится формула: количество шагов ($= \log n + 1$). Это классический пример алгоритма с логарифмической сложностью ($O(\log n)$), где время работы растёт очень медленно по сравнению с ростом объёма данных.

Хотя в этой лекции другие типы не разбираются детально, логика введения O -символики подразумевает, что существуют и иные классы (например, константная ($O(1)$), линейная ($O(n)$), квадратичная ($O(n^2)$) и т.д.), которые будут изучаться далее.



Билет №3: Статические и динамические массивы. Стратегия расширения динамического массива и амортизационный анализ сложности.

1. Массивы в памяти (RAM)

Статический массив — это непрерывный блок ячеек памяти (слайд 3). Его главное преимущество — время доступа к любому элементу. Процессор может вычислить адрес элемента мгновенно, так как знает (слайд 5):

1. Адрес начала массива в памяти (например, `0x1000`).
2. Размер одного элемента (например, 4 байта для `int`).
3. Индекс искомого элемента (i).

Формула вычисления адреса: `Address = Start + i × Size`. Это обеспечивает доступ за константное время — **$O(1)$** . Это и есть статический массив — его размер фиксирован и известен заранее.

2. Динамический массив (`std::vector`, `ArrayList`, `list` в Python)

Проблема статического массива в том, что мы не всегда знаем нужный размер заранее. Динамический массив решает эту задачу, используя две характеристики (слайд 7-8):

- **Size (Размер)**: Количество элементов, которые реально лежат в массиве в данный момент.
- **Capacity (Емкость)**: Количество памяти, выделенной "про запас". Capacity всегда больше или равен Size.

Алгоритм расширения (вставка элемента):

1. Если `Size < Capacity`, то вставка нового элемента происходит просто записью по нужному индексу за **$O(1)$** .
2. Если `Size == Capacity`, то свободного места нет. Происходит **реаллокация**:

- Выделяется новый, больший блок памяти (обычно в 2 раза больше старого).
- Все существующие элементы копируются из старой памяти в новую (это занимает **O(N)** операций, где N — текущий размер).
- Старая память освобождается.
- Новый элемент добавляется уже в новый массив.

Таким образом, большинство вставок дешевые ($O(1)$), но некоторые (при расширении) — дорогие ($O(N)$).

3. Амортизационный анализ сложности

Амортизированная сложность — это **средняя стоимость операции в худшем случае при выполнении последовательности операций** (слайд 10). Это позволяет оценить производительность структуры данных в целом, "распределив" стоимость редких дорогих операций на все остальные.

Аналогия с "Безлимитным проездным" (слайд 10-11):

- Разовая поездка стоит 100 руб. Каждый раз дешево.
- Годовой проездной стоит 18 000 руб. Один раз очень дорого.
- В среднем за день (при ежедневных поездках) проездной обходится ≈ 50 руб., что дешевле разовых поездок.

Так же и с массивом:

- Большинство вставок — "дешевые" ($O(1)$).
- Редкие вставки (расширение) — "дорогие" ($O(N)$).
- В **среднем**, если распределить стоимость расширения на все операции, стоимость одной вставки получается **$O(1)$** .

4. Математика расширения (при стратегии удвоения)

Рассмотрим процесс роста массива от 1 до N элементов со стратегией удвоения емкости (слайды 12-23). Копирование элементов происходит, когда массив расширяется:

- При росте до $N = 2$: копируется 1 элемент.
- При росте до $N = 4$: копируется 2 элемента.
- При росте до $N = 8$: копируется 4 элемента.
- ...
- При последнем расширении до N: копируется $N/2$ элементов.

Общая стоимость всех операций **копирования** за всё время роста до N равна сумме:

$$1+2+4+\dots+N/2 \approx N/2+2+4+\dots+N/2 \approx N$$

Итог за N вставок:

1. **N** операций записи самих элементов.
2. $\approx N$ операций копирования при расширениях.
3. **Суммарная стоимость (Total):** $N+N=2N$

Средняя стоимость одной операции (Average):

$$\text{Average} = \frac{\text{Total}}{N} = \frac{2N}{N} = O(1)$$

Вывод: Благодаря стратегии расширения (обычно удвоения) и амортизационному анализу, мы можем утверждать, что динамический массив обеспечивает вставку элемента в конец за **$O(1)$ амортизированно**.

5. Важный пример-предостережение: Строки в Python (слайды 24-26)

Строки в Python — неизменяемые (`immutable`). Это значит, что операция `s += "a"` не изменяет существующую строку, а создает **новую**, копируя в нее всю старую строку и добавляя один символ.

- Неэффективный код (слайд 24):

```
s = ""
for i in range(N):
    s += "a" # O(len(s)) на каждой итерации!
```

Сложность такого кода: $1+2+\dots+N=O(N^2)$. При больших N (например, 100 000) это будет работать минуты.

- Эффективный код (слайд 25):

```
parts = []
for i in range(N):
    parts.append("a") # O(1) амортизированно для списка
s = "".join(parts) # join считает общий размер и выделяет память один раз
```

Здесь используется динамический массив (``list``) для накопления частей, а затем эффективное склеивание. Сложность такого подхода — $O(N)$.



Билет №4: Связные списки: односвязные, двусвязные и циклические. Сравнение с массивами (преимущества и недостатки).

Введение. Зачем нужны списки?

Массивы, несмотря на свои преимущества, имеют два ключевых недостатка (слайд 2):

1. **Фрагментация памяти:** Массив требует **непрерывного** участка памяти. Это может быть проблематично, если нужно выделить память под большой массив.
2. **Дороговизна изменений:** При вставке или удалении элемента в начало или середину массива приходится сдвигать все последующие элементы, что занимает $O(N)$ операций.

Связные списки решают эти проблемы, жертвуя скоростью доступа к элементу по индексу.

1. Определение и базовые понятия

Связный список — это структура данных, состоящая из узлов (`Node`), каждый из которых хранит данные и ссылку (ссылку) на следующий (и, возможно, предыдущий) узел. Список может быть типизированным или нетипизированным, сортированным или нет (слайд 6). В лекции рассматриваются основные операции АДТ (абстрактного типа данных) списка: проверка на пустоту, добавление в начало/конец/середину, доступ к элементам (слайд 5).

2. Односвязный список (Singly Linked List)

В односвязном списке каждый узел хранит ссылку только на следующий элемент.

- Структура узла (слайд 7):

```
class Node:
    def __init__(self, data: Any, next: Optional['Node'] = None):
        self.data = data
        self.next = None # Ссылка на следующий узел
```

- Структура списка (слайды 8, 10, 11, 13):

Класс списка обычно хранит указатели на первый (`head`) и последний (`tail`) элементы, а также кэширует размер (`_size`) для получения длины за $O(1)$.

```
class SinglyLinkedList:
    def __init__(self):
        self.head: Optional[Node] = None
        self.tail: Optional[Node] = None # Оптимизация для вставки в конец
        self._size: int = 0
```

- **Основные операции и их сложность:**

- **Вставка в начало (push_front)** (слайд 19): Создается новый узел, его `next` указывает на текущую `head`, и `head` обновляется. **O(1)**.
- **Вставка в конец (push_back)** (слайд 20): Благодаря указателю `tail`, новый узел добавляется после него, и `tail` обновляется. **O(1)**.
- **Удаление из начала (pop_front)** (слайд 21): `head` просто перемещается на следующий элемент. **O(1)**.
- **Вставка в середину/по индексу (insert)** (слайды 22-24): Требуется сначала дойти до нужной позиции (итерация от `head`), что занимает **O(N)**, затем вставка за **O(1)**.
- **Удаление по значению (remove)** (слайды 25, 27): В худшем случае нужно пройти весь список в поисках элемента. **O(N)**.
- **Доступ по индексу (__getitem__)** (слайды 14-18): Нет прямого доступа. Чтобы получить *i*-й элемент, нужно пройти *i* шагов от головы. **O(N)**.

3. Двусвязный список (Doubly Linked List)

В двусвязном списке каждый узел хранит ссылки как на следующий (`next`), так и на предыдущий (`prev`) узел, что позволяет перемещаться по списку в обе стороны (слайд 28).

- **Структура узла (слайд 29):**

```
class Node:
    def __init__(self, data: Any):
        self.data = data
        self.next: Optional['Node'] = None
        self.prev: Optional['Node'] = None # Ссылка на предыдущий узел
```

- **Структура списка (слайды 30-33):** Аналогична односвязному, но с возможностью обратного прохода (слайд 34-35).

```
def __reversed__(self) -> Iterator[Any]:
    current = self.tail
    while current:
        yield current.data
        current = current.prev
```

- **Основные операции и их сложность:**

- **Вставка в начало (push_front)** (слайд 39): **O(1)**. Нужно также обновить `prev` у старой головы.
- **Вставка в конец (push_back)** (слайд 40): **O(1)**. Аналогично обновляется `prev` у нового хвоста.
- **Удаление из начала (pop_front)** (слайд 41-42, 44): **O(1)**. Новая голова должна обнулить свой `prev`.
- **Удаление из конца (pop_back)** (слайд 46): **O(1)**. Благодаря указателю `tail` и связи `prev`.
- **Доступ по индексу (__getitem__)** (слайды 37-38): В двусвязном списке можно применить **оптимизацию**: если искомый индекс меньше `_size//2`, то идем от `head`, иначе — от `tail`. Это сокращает путь в среднем в 2 раза, но асимптотически сложность остается **O(N)**.

4. Циклический список (Circular Linked List)

Это вариация связного списка (как правило, односвязного или двусвязного), в котором последний узел (`tail`) ссылается не на `None`, а на первый узел (`head`) (слайд 51). Таким образом, список замыкается в кольцо.

- **Ключевая особенность:** `Tail.next = Head`.

- **Применение:** Классический пример — **Round-Robin планировщик** в операционных системах, где каждому процессу по очереди выделяется квант времени, и после последнего процесса управление снова передается первому (слайд 51).

5. Сравнение с массивами. Преимущества и недостатки.

На слайдах 55-57 приведена итоговая сравнительная таблица и рекомендации по использованию.

Операция / Характеристика	Массив (Динамический, напр. Python list)	Связный список (Linked List)
Доступ по индексу [i]	$O(1)$ $O(1)$ (мгновенно, прямая адресация)	$O(N)$ $O(N)$ (нужно пройти от начала)
Вставка в начало	$O(N)$ $O(N)$ (требуется сдвиг всех элементов)	$O(1)$ $O(1)$ (достаточно перекинуть ссылки)
Вставка в середину	$O(N)$ $O(N)$ (сдвиг части элементов)	$O(1)$ $O(1)$ (если уже есть указатель на место вставки)
Использование памяти	Компактно (только данные), но требуется цельный блок памяти.	Фрагментировано (узлы могут быть разбросаны по RAM). Требуется дополнительная память для хранения ссылок (next/prev).
Кэш процессора	Отлично (данные лежат подряд, эффективно предвыбираются)	Плохо (процессор "прыгает" по случайным адресам)

Вывод и рекомендации из лекции (слайды 56-57):

1. По умолчанию всегда используйте массив (например, `list` в Python). Это простая, быстрая и кэш-дружелюбная структура.
2. Используйте связный список только в специфических случаях:
 - Если вам нужно **очень много вставок и удалений в начало или середину** списка.
 - Если при этом вам **не нужен доступ по индексу** (или он нужен крайне редко).

Примеры из реальной жизни (слайды 59-62):

- **Undo/Redo (Ctrl+Z/Ctrl+Y)** в графических редакторах.
- **Музыкальные плейлисты** (переключение между треками вперед/назад — идеально для двусвязного списка).
- **Блокчейн (Bitcoin)** — каждый блок содержит хеш предыдущего, что делает его буквально односвязным списком.
- Реализация **хеш-таблиц** (для разрешения коллизий методом цепочек).



Билет №5: Абстрактные типы данных: Стек, Очередь, Дек. Принципы работы и примеры практического использования.

1. Понятие абстрактного типа данных (АТД)

- **Абстрактный тип данных (АТД)** — это модель, которая определяет набор значений и набор операций над этими значениями, но скрывает детали конкретной реализации.
- Это можно представить как **"контракт"** или интерфейс. Мы описываем, что структура умеет делать (например, "вставить элемент", "удалить элемент"), но не как именно это реализовано внутри.
- **Интерфейс** — это заранее определенный набор доступных операций.
- **Реализация** — это конкретный код, определяющий, как данные хранятся в памяти (массив, связный список, дерево) и как операции выполняются.

Это подобно пульту управления: пользователю важно, что есть кнопки, а не то, как устроена электроника внутри.

2. Стек (Stack)

Принцип работы:

Стек (от англ. stack — стопка) — это АТД, работающий по принципу **LIFO (Last-In, First-Out)**, что означает «**последним пришёл — первым вышел**» (слайд 4). Представьте стопку тарелок: новая тарелка кладется сверху, и взять всегда можно только верхнюю.

- **Конец для операций** называется **вершиной стека (top)**.
- **Основные операции (интерфейс):**
 - `push` — добавить элемент на вершину стека (слайд 4-5).
 - `pop` — удалить и вернуть элемент с вершины стека (слайд 4-5).
 - `peek` или `top` — посмотреть элемент на вершине без его удаления (слайд 4).
 - `empty` — проверить, пуст ли стек (слайд 4).

Реализация в Python:

В Python нет отдельного класса `Stack`, но для этой цели отлично подходит обычный список `list` (слайд 6). Мы используем конец списка как вершину стека.

```
stack = []
stack.append("A") # push -> O(1)
stack.append("B")
top = stack[-1]   # peek -> O(1)
item = stack.pop() # pop -> O(1) # item = "B"
```

Примеры практического использования (слайд 7):

1. **Реализация рекурсии:** Любой рекурсивный вызов в компьютере реализуется через стек вызовов (call stack), где хранятся адреса возврата и локальные переменные функций.
2. **Вычисление постфиксных выражений (обратная польская запись):** Калькуляторы используют стек для вычисления выражений типа `2 3 + 5 *`.
3. **Хранение истории:** Например, история запросов в браузере (кнопка "Назад") или история изменений (Undo/Redo) в текстовых редакторах и графических редакторах.

3. Очередь (Queue)

Принцип работы:

Очередь — это АТД, работающий по принципу **FIFO (First-In, First-Out)**, что означает «**первым пришёл — первым вышел**» (слайд 8). Как обычная очередь в магазине: первый покупатель в очереди обслуживается первым и покидает её.

- У очереди есть два конца (слайд 8-9):
 - **Хвост (tail)** — конец, куда новые элементы ставятся (`push`).
 - **Голова (head)** — начало, откуда элементы извлекаются (`pop`).
- **Основные операции (интерфейс):**
 - `push` (или `enqueue`) — вставить элемент в хвост очереди (слайд 8).
 - `pop` (или `dequeue`) — удалить и вернуть элемент из головы очереди (слайд 8).
 - `empty` — проверка на пустоту (слайд 8).
 - `size` — получение количества элементов (слайд 8).

Реализация в Python:

Использовать список `list` для реализации очереди — **плохая идея**. Удаление первого элемента (`pop(0)`) требует сдвига всех остальных элементов, что занимает **O(N)** времени (слайд 10-11).

Правильный способ — использовать `collections.deque` (двусторонняя очередь), которая оптимизирована для быстрых вставок и удалений с обоих концов (слайд 12).

```
from collections import deque
q = deque()
```



```
q.append(1) # push в хвост -> O(1)
val = q.popleft() # pop из головы -> O(1)
```

Примеры практического использования (слайд 13):

1. **Управление доступом к ресурсу:** Очередь задач на принтере или пул потоков в многопоточном программировании.
2. **Управление потоками:** Планировщик задач в операционной системе часто использует очереди для распределения процессорного времени.
3. **Буферизация данных:** Например, буфер ввода-вывода, где данные накапливаются перед отправкой или записью.

4. Дек (Deque — Double-Ended Queue)

Принцип работы:

Дек (двусторонняя очередь) — это АТД, который позволяет эффективно добавлять и удалять элементы **с обоих концов** (слайд 14). Он объединяет в себе возможности стека и очереди, поддерживая как LIFO, так и FIFO операции.

- **Основные операции (интерфейс):**
 - `pushBack` — вставить элемент в конец.
 - `popBack` — удалить элемент с конца.
 - `pushFront` — вставить элемент в начало.
 - `popFront` — удалить элемент с начала.
 - `empty` — проверка на пустоту.

Реализация в Python:

В Python уже есть готовая и оптимизированная реализация — `collections.deque` (слайд 15).

```
from collections import deque
d = deque([1, 2, 3])
d.append(4) # pushBack -> [1, 2, 3, 4]
d.appendleft(0) # pushFront -> [0, 1, 2, 3, 4]
d.pop() # popBack -> удалит 4
d.popleft() # popFront -> удалит 0
```

Примеры практического использования:

- Реализация стека и очереди "в одном флаконе".
- Алгоритмы, требующие хранения истории с обеих сторон (например, некоторые алгоритмы поиска).
- Задачи на скользящее окно (sliding window), где нужно быстро получать доступ к элементам в начале и конце окна.

5. Сводная таблица (слайд 17)

АТД	Реализация в Python (рекомендуемая)	Добавление	Удаление	Доступ / Поиск
Стек	<code>list</code>	<code>append()</code> — O(1)	<code>pop()</code> — O(1)	<code>[-1]</code> (Top) — O(1)
Очередь	<code>collections.deque</code>	<code>append()</code> — O(1)	<code>popleft()</code> — O(1)	<code>[0]</code> (Front) — O(1)
Дек	<code>collections.deque</code>	<code>append()</code> / <code>appendleft()</code> — O(1)	<code>pop()</code> / <code>popleft()</code> — O(1)	<code>[0]</code> / <code>[-1]</code> (Ends) — O(1)



Билет №6: Хеширование. Полиномиальный хеш и его применение.

1. Хеширование (Hashing)

Определение: Хеширование — это класс методов, который позволяет преобразовывать исходные данные произвольного размера (ключ) в число фиксированного размера — **хеш-код** — с помощью специальной **хеш-функции**. Этот хеш-код затем используется для ускорения поиска, выступая в роли индекса в памяти, что в идеале дает доступ за **$O(1)$** (слайд 2).

Коллизии: Важно понимать, что хеш-функция не гарантирует уникальности, так как область возможных значений хеш-функции меньше, чем область возможных ключей. Ситуация, когда два разных ключа дают одинаковый хеш-код, называется **коллизией**. Хорошая хеш-функция должна минимизировать вероятность коллизий (слайд 2).

2. Полиномиальный хеш (Polynomial Hash)

Это один из самых популярных способов хеширования строк (слайд 3).

Определение: Пусть дана строка $s[0..n-1]$. Её полиномиальный хеш вычисляется по формуле:

$$h = hash(s) = s[0] * p^0 + s[1] * p^1 + \dots + s[n-1] * p^{(n-1)}$$

где:

- p — некоторое простое число (основание степени), которое выбирают больше размера алфавита (например, 31 или 53 для латиницы).
- $s[i]$ — числовой код i -го символа (например, его порядковый номер в алфавите или Unicode-код).

Пример (слайд 4): Для строки $s = \text{"sirius"}$ и основания $p = 3$ хеш вычисляется как сумма произведений кодов символов на соответствующие степени тройки.

Важное замечание: Чтобы избежать работы с огромными числами, на практике хеш считают по модулю m , где m часто выбирают как 2^{64} (используя естественное переполнение целочисленных типов) или 2^{32} (слайд 3).

Билет №7: Хеш-таблицы. Разрешение коллизий методами цепочек и открытой адресации. Рехеширование.

1. Хеш-таблица (Hash Table)

Определение: Хеш-таблица — это структура данных, реализующая интерфейс ассоциативного массива (словаря). Она позволяет хранить пары (ключ, значение) и выполнять три основные операции: **добавление**, **поиск** и **удаление** по ключу (слайд 13).

Принцип работы: Операция начинается с вычисления хеш-кода ключа с помощью хеш-функции: $i = h(\text{key})$. Полученное число i используется как индекс в массиве n , где должна храниться или искаться нужная пара (слайд 13). Проблема коллизий решается одним из методов, описанных ниже.

2. Разрешение коллизий методом цепочек (Chaining)

Идея: Каждая ячейка массива n (называемая "ведром" или "basket") хранит не одно значение, а указатель на начало **связного списка**. Все элементы, у которых хеш-код равен i , добавляются в список, находящийся в ячейке i (слайд 15-16).

Пример работы (слайд 17):

Допустим, размер таблицы = 5.

- $\text{put}(\text{"cat"}, 10) : \text{hash}(\text{"cat"}) \% 5 = 3$. В `buckets[3]` добавляется пара `(\text{"cat"}, 10)`.
- $\text{put}(\text{"dog"}, 20) : \text{hash}(\text{"dog"}) \% 5 = 3$ (коллизия!). В `buckets[3]` добавляется пара `(\text{"dog"}, 20)`. Теперь в этом "ведре" список из двух элементов.
- $\text{get}(\text{"dog"})$: Идем в `buckets[3]` и проходим по списку, пока не найдем ключ `\text{"dog"}`.

Сложность (слайды 18-19):

- Вставка:** В худшем случае **$O(1)$** (если не проверять уникальность ключей и просто добавлять в начало/конец списка). Если нужно проверять уникальность, то в худшем случае **$O(L)$** , где L — длина цепочки.

- **Поиск/Удаление:** В худшем случае пропорциональна длине цепочки $O(L)$. В самом плохом случае (если все ключи попали в одну ячейку) поиск может деградировать до $O(n)$.
- **Удаление:** Если есть ссылка на удаляемый элемент в двусвязном списке — $O(1)$. Если удаление по ключу — $O(L)$.

3. Разрешение коллизий методом открытой адресации (Open Addressing)

Идея: Все элементы хранятся непосредственно в ячейках массива N . Если ячейка с индексом $i = h(key)$ занята, мы начинаем искать другую свободную ячейку по определенному правилу (слайд 30).

Методы открытой адресации (слайды 31-33):

1. **Линейное пробирование (Linear Probing):** При коллизии последовательно проверяем ячейки $i+1, i+2, i+3, \dots$ (по модулю размера таблицы) до нахождения свободной. Шаг фиксирован и равен 1.
2. **Линейный поиск с шагом q :** Вариация линейного пробирования, где шаг — фиксированное число q (слайд 32).
3. **Квадратичное пробирование (Quadratic Probing):** Шаг поиска изменяется квадратично: $i+1, i+4, i+9, \dots$. Это помогает избежать кластеризации, характерной для линейного пробирования (слайд 33).

Проблема удаления (слайд 31): При открытой адресации нельзя просто очистить ячейку, так как это разорвет цепочку поиска. Вместо этого используются специальные маркеры (tombstones), помечающие ячейку как "удалено".

4. Коэффициент заполнения и Рехеширование (Resize)

Чтобы операции с хеш-таблицей оставались эффективными, важно контролировать её заполненность.

- **Коэффициент заполнения (Load Factor) α :** Отношение количества хранимых элементов к размеру таблицы (слайд 28).

$$\alpha = \frac{\text{Кол-во элементов}}{\text{Размер таблицы}}$$
- **Рехеширование (Resize):** Когда α превышает определенный порог (например, **0.7 для метода цепочек** или **0.5 для открытой адресации**), производится увеличение размера таблицы (обычно в 2 раза). Процесс включает (слайд 28-29):
 1. Создание нового массива (большого размера).
 2. Перебор всех существующих элементов.
 3. **Перехеширование:** Для каждого элемента заново вычисляется индекс по формуле $\text{hash}(key) \% \text{new_size}$ (так как размер изменился, индекс тоже изменится), и элемент вставляется в новую таблицу.

Это дорогая операция ($O(n)$), но она происходит редко, что позволяет сохранять амортизированную сложность операций близкой к $O(1)$.



Билет №8: Бинарный поиск в отсортированном массиве и его вариации (Lower/Upper Bound).

Бинарный поиск по ответу.

1. Задача поиска

Поиск — это операция нахождения местоположения элемента с заданным значением ключа в наборе данных (слайд 2, 4). Тип данных (упорядоченные или нет) определяет стратегию поиска (слайд 5).

2. Бинарный поиск (Binary Search)

Условие применимости: Массив должен быть **отсортирован** (слайд 10).

Алгоритм (слайд 10, 12):

1. Установить границы поиска: левая $l = 0$, правая $r = n - 1$.
2. Пока $l \leq r$:
 - Найти индекс среднего элемента: $m = l + (r - l) // 2$ (такая запись помогает избежать переполнения при сложении больших чисел).

- Если `arr[m] == target` — поиск завершен, возвращаем `m`.
- Если `target < arr[m]` — искомый элемент в левой половине, сдвигаем правую границу: `r = m - 1`.
- Если `target > arr[m]` — искомый элемент в правой половине, сдвигаем левую границу: `l = m + 1`.

3. Если цикл завершился и элемент не найден, вернуть `-1`.

Сложность: $O(\log n)$. На каждой итерации область поиска сужается вдвое.

3. Вариации: Lower Bound и Upper Bound

В отсортированных массивах с дубликатами часто нужно найти не любое вхождение, а границы диапазона значений (слайд 20).

- **Lower Bound (нижняя граница):** Индекс первого элемента, который не меньше ($\geq$) заданного значения.
 - Пример для массива `[1, 2, 2, 2, 5]` и `target = 2`: `lower bound = 1` (первая двойка).
 - Реализация (слайды 21-27): Границы `l = 0`, `r = len(arr)`. Цикл `while l < r`. Если `arr[m] < target`, то нужный элемент правее (`l = m + 1`). Иначе (если `arr[m] >= target`) сдвигаем левую границу к `m` (`r = m`). Возвращается `l`.
- **Upper Bound (верхняя граница):** Индекс первого элемента, который строго больше ($>$) заданного значения.
 - Пример для `[1, 2, 2, 2, 5]` и `target = 2`: `upper bound = 4` (элемент 5).
 - Реализация: аналогична Lower Bound, но условие меняется на `if arr[m] <= target: l = m + 1 else: r = m`.

4. Бинарный поиск по ответу

Это мощный подход, когда нам нужно найти **граничное значение**, при котором меняется состояние системы с "можно" на "нельзя", и это изменение монотонно (слайд 28).

Условия применимости:

1. Есть некоторая функция проверки `check(x)`, которая монотонна (например, для всех $x < X$ она возвращает `True`, а для $x \geq X$ — `False`, или наоборот).
2. Мы ищем это граничное значение `x`.

Классический пример: Задача "Коровы в стойла" (слайды 28-36)

- **Условие:** На прямой расположены `N` стойл с известными координатами. Нужно расставить `K` коров так, чтобы минимальное расстояние между любыми двумя коровами было **максимально возможным**.
- **Применение бинарного поиска по ответу:**
 1. Мы не ищем корову. Мы ищем **расстояние `D`**.
 2. **Диапазон поиска:** от `l = 0` до `r = max_coord - min_coord`.
 3. **Функция проверки `check(D)`:** Можно ли расставить `K` коров так, чтобы расстояние между любыми двумя было **хотя бы `D`**? Это делается "жадно": ставим первую корову в первое стойло, затем ищем следующее стойло, отстоящее от текущего на расстояние $\geq D$, и так далее. Если удалось поставить всех `K` коров, возвращаем `True`, иначе `False`. Сложность проверки **$O(N)$** .
 4. **Бинарный поиск:**
 - Если `check(D)` вернула `True` — расстояние `D` достижимо, но, возможно, можно сделать его еще больше. Запоминаем `D` как кандидат в ответ (`ans = D`) и пробуем увеличить расстояние (`l = m + 1`).
 - Если `check(D)` вернула `False` — расстояние слишком большое, его нужно уменьшить (`r = m - 1`).

Сложность: $O(N \log M)$, где `M` — диапазон поиска.



Билет №9: Продвинутые методы поиска: интерполяционный, экспоненциальный и тернарный поиск.

1. Интерполяционный поиск (Interpolation Search)

Это улучшение бинарного поиска для случаев, когда данные в массиве не только отсортированы, но и **равномерно распределены** (слайд 38).

Идея: Вместо того чтобы всегда брать средний элемент, мы используем формулу линейной интерполяции для оценки наиболее вероятной позиции искомого элемента. Это похоже на поиск слова в бумажном словаре: если мы ищем слово на букву "Б", мы откроем книгу ближе к началу, а не строго посередине (слайд 46).

Формула для оценки позиции (слайд 38):

$$pos = l + \frac{(target - arr[l]) \cdot (r - l)}{arr[r] - arr[l]}$$

Алгоритм (слайды 39-45):

1. Пока $l \leq r$ и `target` находится в пределах `[arr[l], arr[r]]`.
2. Вычислить `pos` по формуле.
3. Сравнить `arr[pos]` с `target` и сузить границы аналогично бинарному поиску.

Сложность:

- **В среднем:** $O(\log \log n)$ — очень быстро.
- **В худшем случае:** $O(n)$ — если данные распределены крайне неравномерно (например, экспоненциально).

2. Экспоненциальный поиск (Exponential Search)

Применяется для поиска в отсортированных массивах, особенно когда правая граница неизвестна или массив потенциально бесконечен (слайд 47).

Алгоритм (слайды 47, 49-50):

1. Начинаем с `step = 1`.
2. Пока текущий элемент `arr[step]` меньше искомого `target`, удваиваем шаг (`step *= 2`). Таким образом, мы быстро находим диапазон `[step/2, step]`, в котором гарантированно находится искомый элемент (или выходим за границы массива).
3. Запускаем обычный бинарный поиск в найденном диапазоне.

Сложность: $O(\log i)$, где i — позиция искомого элемента. Алгоритм очень эффективен, когда искомый элемент находится близко к началу массива.

3. Тернарный поиск (Ternary Search)

Используется для поиска экстремума (минимума или максимума) **униmodalьной функции** на отрезке (слайд 53).

Униmodalьная функция — это функция, которая сначала строго возрастает (или убывает), а потом строго убывает (или возрастает).

Идея: Делим отрезок `[l, r]` на **три равные части** двумя точками `m1` и `m2` (слайд 53-54).

- $m1 = l + (r - l) / 3$
- $m2 = r - (r - l) / 3$

Алгоритм поиска минимума (слайд 54-58):

Пока длина отрезка больше заданной точности `eps`:

1. Вычисляем `f(m1)` и `f(m2)`.
2. Если `f(m1) < f(m2)`, то минимум не может находиться правее `m2` (так как функция униmodalьна). Отбрасываем правую часть: `r = m2`.
3. Иначе (если `f(m1) >= f(m2)`), минимум не может находиться левее `m1`. Отбрасываем левую часть: `l = m1`.

Сложность: $O(\log n)$. На каждой итерации отрезок сокращается в 1.5 раза.

Преимущества (слайд 59): Не требует вычисления производной, работает для любых унимодальных функций.

4. Метод золотого сечения (Golden Section Search)

Это оптимизация тернарного поиска (слайд 60). Точки деления выбираются не равномерно, а в пропорциях **золотого сечения** $\phi \approx 1.618$. Благодаря этому свойству, на каждой итерации (после первой) одна из старых точек деления совпадает с новой, что позволяет вычислять функцию только **один раз** вместо двух.

Сложность: $O(\log n)$, но с вдвое меньшей константой по количеству вычислений функции по сравнению с тернарным поиском.



Билет №10: Поиск k-й порядковой статистики (алгоритм QuickSelect).

Задача: Найти k -й по величине элемент в неотсортированном массиве (например, 3-й минимальный). k -я порядковая статистика — это элемент, который окажется на позиции k (при 0-индексации) в отсортированном массиве.

Идея алгоритма QuickSelect (слайд 67):

Алгоритм основан на подходе "разделяй и властвуй", используемом в быстрой сортировке (QuickSort). Он рекурсивно обрабатывает только ту часть массива, где находится искомая статистика.

Алгоритм (слайды 68-75):

- Базовый случай:** Если в массиве один элемент, вернуть его.
- Выбор опорного элемента (pivot):** Выбирается случайный элемент массива (для улучшения среднего времени работы).
- Разделение (Partition):** Массив делится на три части (слайд 71):
 - `lows`: все элементы, **меньшие** `pivot`.
 - `pivots`: все элементы, **равные** `pivot`.
 - `highs`: все элементы, **большие** `pivot`.
- Рекурсивный выбор:**
 - Если $k < \text{len}(\text{lows})$ (искомая статистика находится среди меньших элементов), рекурсивно вызываем `quick_select(lows, k)`.
 - Если $k < \text{len}(\text{lows}) + \text{len}(\text{pivots})$ (искомая статистика попадает на элемент(ы), равный `pivot`), возвращаем `pivot`.
 - Иначе (искомая статистика находится среди больших элементов), рекурсивно вызываем `quick_select(highs, k - \text{len}(\text{lows}) - \text{len}(\text{pivots}))` (корректируем k , так как мы отбросили `lows` и `pivots`).

Сложность:

- В среднем: $O(n)$** . Благодаря случайному выбору `pivot`, алгоритм работает очень быстро на практике.
- В худшем случае: $O(n^2)$** (например, если всегда выбирать минимальный элемент в качестве `pivot`, и искать максимальный).



Билет №11: Квадратичные сортировки (пузырьком, вставками, выбором). Анализ устойчивости и использования памяти.

1. Введение в сортировку

Сортировка — это алгоритм для упорядочивания элементов в списке по возрастанию или убыванию (слайд 2).

Ключевыми свойствами алгоритмов являются (слайд 6):

- Устойчивость (stability):** Равные элементы сохраняют свой относительный порядок после сортировки.

- **Сортировка на месте (in-place):** Алгоритм не требует дополнительной памяти (или требует $O(1)$ дополнительной памяти).

2. Квадратичные сортировки ($O(n^2)$)

Эти алгоритмы просты в реализации, но неэффективны на больших массивах. Подходят для обучения или работы с очень маленькими данными (слайд 7).

А. Сортировка пузырьком (Bubble Sort)

- **Принцип (слайд 10-11):** Алгоритм проходит по массиву, сравнивая соседние элементы. Если порядок неверный, они меняются местами. Самые "тяжелые" элементы "всплывают" в конец массива. Проходы повторяются, пока массив не будет отсортирован.
- **Реализация с оптимизацией (слайд 14-15):** Вводится флаг `swapped`. Если за проход не было ни одной замены, массив уже отсортирован, и алгоритм завершается досрочно. Это дает сложность $O(n)$ в лучшем случае (на уже отсортированном массиве).
- **Характеристики:**
 - Время: $O(n^2)$ в среднем и худшем, $O(n)$ в лучшем.
 - Память: $O(1)$ (in-place).
 - Устойчивость: **Да** (обмен соседних элементов не нарушает порядок равных).

Б. Сортировка вставками (Insertion Sort)

- **Принцип (слайд 17-18):** Массив мысленно делится на отсортированную и неотсортированную части. На каждом шаге элемент из неотсортированной части (`key`) вставляется на правильное место в отсортированной части путем сдвига больших элементов вправо.
- **Реализация (слайд 21-22):** Цикл от 1 до $n-1$. Для каждого `i` запоминаем `arr[i]` и сдвигаем элементы `arr[i-1]`, `arr[i-2]`, ... вправо, пока они больше `key`.
- **Характеристики:**
 - Время: $O(n^2)$ в среднем и худшем, $O(n)$ в лучшем (на почти отсортированных массивах). Это ее ключевое преимущество.
 - Память: $O(1)$ (in-place).
 - Устойчивость: **Да**.
- **Применение (слайд 22):** Эффективна на маленьких ($n < 50$) и почти отсортированных массивах. Используется внутри сложных гибридных алгоритмов, таких как Timsort.

В. Сортировка выбором (Selection Sort)

- **Принцип (слайд 23, 25):** Массив делится на отсортированную и неотсортированную части. На каждом шаге в неотсортированной части ищется **минимальный** элемент и меняется местами с первым элементом этой части.
- **Реализация (слайд 29):** Два вложенных цикла: внешний для прохода по границе, внутренний для поиска минимума.
- **Характеристики:**
 - Время: $O(n^2)$ **всегда** (даже на отсортированном массиве, так как поиск минимума требует полного прохода).
 - Память: $O(1)$ (in-place).
 - Устойчивость: **Нет**. Обмен может нарушить порядок равных элементов (слайд 28).



Билет №12: Метод «Разделяй и властвуй». Сортировка слиянием (Merge Sort): рекурсивная и итеративная реализация.

1. Метод «Разделяй и властвуй» (Divide and Conquer)

Это парадигма разработки алгоритмов, состоящая из трех этапов (слайды 32-36):

1. **Разделение (Divide):** Исходная задача разбивается на несколько подзадач того же типа, но меньшего размера.
2. **Властвование (Conquer):** Подзадачи решаются рекурсивно (или тривиально, если размер мал).
3. **Комбинирование (Combine):** Решения подзадач объединяются для получения ответа к исходной задаче.

- **Примеры:** Бинарный поиск, сортировка слиянием, быстрая сортировка.

2. Сортировка слиянием (Merge Sort)

Классический алгоритм, основанный на методе "разделяй и властвуй" (слайд 40).

Алгоритм (слайд 42-43):

1. **Разделение:** Массив делится на две равные (или почти равные) половины.
2. **Властвование:** Каждая половина сортируется рекурсивно с помощью merge sort.
3. **Комбинирование (Слияние):** Два отсортированных подмассива объединяются в один отсортированный массив.

Процесс слияния (Merge) (слайд 45):

- Создается новый массив `result`.
- Два указателя (`i`, `j`) проходят по сравниваемым элементам левого и правого подмассивов.
- Меньший элемент копируется в `result`, соответствующий указатель сдвигается.
- Остатки одного из подмассивов копируются в конец `result`.

Реализации:

1. Рекурсивная (Top-down) (слайд 46-47):

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

2. Итеративная (Bottom-up) (слайд 48-49):

- Не использует рекурсию. Слияние происходит снизу вверх.
- Начинается с подмассивов размера 1, затем сливает их в подмассивы размера 2, затем 4 и так далее, пока не получится целый массив.
- Управляется переменной `step` (размер сливаемых подмассивов).

Характеристики (слайд 47):

- **Время:** $O(n \log n)$ во всех случаях (лучшем, среднем, худшем).
- **Память:** $O(n)$ (требуется дополнительный массив для слияния).
- **Устойчивость:** **Да** (при слиянии, если элемент слева $\leq$ элементу справа, берется сначала левый, что сохраняет порядок).



Билет №13: Быстрая сортировка (Quick Sort). Стратегии выбора опорного элемента и анализ сложности в среднем и худшем случаях.

1. Быстрая сортировка (Quick Sort)

Алгоритм, разработанный Тони Хоаром в 1960 году. Один из самых быстрых универсальных алгоритмов на практике (слайд 51).

Алгоритм (слайд 53):

1. **Выбор опорного элемента (pivot):** Из массива выбирается некоторый элемент.
2. **Разбиение (Partition):** Массив перестраивается так, чтобы все элементы **меньше** pivot оказались слева от него, а все элементы **больше** — справа. После этого pivot находится на своей окончательной позиции.
3. **Рекурсия:** Рекурсивно применить те же шаги к левой и правой частям.

2. Разбиение (Partition)

В лекции рассматривается схема Ломута (Lomuto partition scheme) (слайд 56-57):

- Pivot выбирается как последний элемент (`arr[high]`).
- Указатель `i` хранит границу элементов, меньших pivot.
- Цикл `j` от `low` до `high-1`: если `arr[j] <= pivot`, то `i` увеличивается, и элементы `arr[i]` и `arr[j]` меняются местами.
- В конце pivot (`arr[high]`) меняется с `arr[i+1]`, и возвращается индекс pivot.

Также упоминается схема Хоара (Hoare partition) (слайд 64-65), которая в среднем выполняет меньше обменов.

3. Стратегии выбора опорного элемента (pivot)

Выбор pivot критически влияет на производительность (слайд 59-62):

1. **Первый/последний элемент:** Просто, но приводит к **худшему случаю $O(n^2)$** на уже отсортированных или почти отсортированных массивах.
2. **Случайный элемент:** Снижает вероятность возникновения худшего случая до практически нулевой. Самый популярный подход.
3. **Медиана трёх (median of three):** Берется медиана из первого, среднего и последнего элементов. Это улучшает выбор на частично упорядоченных данных.

4. Анализ сложности

- **Средний случай: $O(n \log n)$.** Благодаря сбалансированному разбиению.
- **Худший случай: $O(n^2)$.** Возникает, когда разбиение каждый раз получается максимально несбалансированным (например, pivot всегда минимальный или максимальный). Это может произойти при неудачном выборе pivot (например, всегда первый элемент на отсортированном массиве).
- **Память: $O(\log n)$** в среднем (для хранения стека рекурсивных вызовов).
- **Устойчивость: Нет** (стандартная версия). Из-за дальних обменов относительный порядок равных элементов может быть нарушен.



Билет №14: Современные гибридные подходы: обзор алгоритма Timsort (Runs, Galloping).

1. Timsort: История и Идея

Timsort — это гибридный алгоритм сортировки, созданный Тимом Питерсом в 2002 году специально для языка Python (слайд 84). Сегодня это стандартный алгоритм сортировки в Python (`list.sort`, `sorted`), Java (для объектов), JavaScript (V8) и других платформах (слайд 87).

Главная цель: Максимальная эффективность на **реальных данных**, которые часто содержат уже упорядоченные подпоследовательности (слайд 86).

Основа: Гибрид сортировки слиянием (Merge Sort) и сортировки вставками (Insertion Sort) (слайд 85).

Сложность (слайд 88):

- Лучшее время: **$O(n)$** (на почти отсортированных данных).
- Среднее/Худшее время: **$O(n \log n)$.**
- Память: **$O(n)$.**

- Устойчивость: **Да**.

2. Ключевые концепции Timsort

A. Runs (Сегменты) (слайд 90-91)

- Алгоритм разбивает массив на сегменты, называемые **run'ами** — это уже упорядоченные (возрастающие или строго убывающие) подпоследовательности.
- **MinRun**: Минимальная длина такого сегмента (обычно от 32 до 64). Выбирается так, чтобы размер массива, деленный на MinRun, был близок к степени двойки для эффективного слияния.
- Если найденный естественный run короче MinRun, он "добирает" следующие элементы и сортирует их с помощью **сортировки вставками** (которая эффективна на маленьких массивах).

Б. Стек и слияние (слайд 92-95)

- Найденные run'ы помещаются в стек. Длина каждого run'a известна.
- Чтобы слияние было сбалансированным (как в merge sort), поддерживаются **инварианты стека** (правила Тима Питерса). Пусть X, Y, Z — длины трех верхних сегментов:
[1. $X > Y + Z \setminus 2$. 2. $Y > Z$]
- Если правила нарушаются, Y сливается с меньшим из X или Z . Это гарантирует, что размеры сегментов в стеке растут экспоненциально, делая процесс похожим на merge sort.

В. Режим галопа (Galloping Mode) (слайд 96-99)

- При слиянии двух сегментов A и B Timsort отслеживает, сколько раз подряд элемент выбирается из одного и того же сегмента.
- **Порог (Threshold)**: Если из сегмента A выбрано более 7 элементов подряд, алгоритм переходит в режим галопа.
- В этом режиме вместо поштучного сравнения используется **бинарный поиск**, чтобы найти диапазон элементов из A , которые меньше текущего элемента из B . Это позволяет скопировать целый блок за одну операцию, что очень эффективно, когда данные идут крупными блоками (например, при слиянии двух отсортированных баз данных).



Билет №15: Графы: основные определения и способы представления (матрица смежности, списки смежности, список ребер).

1. Зачем нужны графы?

Графы — это универсальный язык для описания связей между объектами (слайд 2). Они применяются повсеместно (слайды 3-7):

- **Социальные сети**: пользователи (вершины), дружба (рёбра).
- **Карты и навигация**: перекрёстки (вершины), дороги (рёбра).
- **Интернет**: веб-страницы (вершины), гиперссылки (рёбра).
- **Программирование**: зависимости пакетов, граф вызовов функций.

2. Основные определения

Граф ($G = (V, E)$) — это совокупность двух множеств (слайд 8):

- (V) (Vertices) — непустое множество **вершин** (узлов).
- (E) (Edges) — множество **рёбер** (связей), соединяющих пары вершин.

Типы графов:

- **Неориентированный граф**: Рёбра не имеют направления. Связь (u, v) симметрична (слайд 9).
- **Ориентированный граф (орграф)**: Рёбра (дуги) имеют направление от (u) к (v) (слайд 9).

- **Взвешенный граф:** Каждому ребру присвоено число — **вес** (расстояние, стоимость) (слайд 11, 13).
- **Простой граф:** Нет петель и кратных рёбер (слайд 14).

Базовые понятия (слайды 16-21):

- **Смежные вершины:** Две вершины, соединённые ребром.
- **Инцидентность:** Ребро (u, v) инцидентно вершинам (u) и (v) .
- **Степень вершины (degree):** Количество рёбер, выходящих из вершины. Для орграфов различают **полустепень захода (in-degree)** и **полустепень исхода (out-degree)**.
- **Путь:** Последовательность вершин, где каждая следующая смежна с предыдущей.
- **Цикл:** Путь, в котором начальная и конечная вершины совпадают.

3. Способы представления графов

Существует три основных способа хранения графа в памяти компьютера.

А. Список рёбер (Edge List)

- **Описание:** Простейший способ. Граф хранится как список всех его рёбер (слайд 23-25).
- **Для неориентированного графа:** Хранятся пары вершин (u, v) .
- **Для ориентированного графа:** Хранятся пары $(from, to)$, где порядок важен.
- **Для взвешенного графа:** Хранятся тройки $(u, v, weight)$.
- **Память:** $O(E)$ — самый экономный для разреженных графов.
- **Применение:** Удобен для алгоритмов, которые работают со списком рёбер (например, алгоритм Крускала или Беллмана-Форда).

Б. Матрица смежности (Adjacency Matrix)

- **Описание:** Квадратная матрица (A) размера $(|V| \times |V|)$, где элемент $(A[i][j])$ хранит информацию о ребре из вершины (i) в вершину (j) (слайд 26-28).
- **Для неориентированного графа:** Матрица симметрична. $(A[i][j] = 1)$ (или вес), если есть ребро.
- **Для ориентированного графа:** Матрица обычно несимметрична.
- **Для взвешенного графа:** В ячейке хранится вес ребра. Если ребра нет, хранят 0 или $(-\infty)$.
- **Реализация (слайд 29-33):** Двумерный список/массив.
- **Память:** $O(V^2)$ — неэффективно для разреженных графов.
- **Сложность операций (слайд 46):**
 - Проверка наличия ребра (u, v) : $O(1)$.
 - Перебор всех соседей вершины (u) : $O(V)$ (нужно просмотреть всю строку).
 - Добавление/удаление ребра: $O(1)$.

В. Список смежности (Adjacency List)

- **Описание:** Массив (или словарь) списков. Для каждой вершины (i) хранится список её соседей (слайд 34-36).
- **Для неориентированного графа:** Ребро (u, v) добавляется в список для (u) и в список для (v) .
- **Для ориентированного графа:** Ребро $(u \rightarrow v)$ добавляется только в список для (u) .
- **Для взвешенного графа:** В списке хранятся пары $(сосед, вес)$.
- **Реализация (слайд 37-42):** Список списков или словарь.
- **Память:** $O(V + E)$ — оптимально для разреженных графов.
- **Сложность операций (слайд 46):**
 - Проверка наличия ребра (u, v) : $O(\text{degree}(u))$ (нужно пройти по списку соседей).
 - Перебор всех соседей вершины (u) : $O(\text{degree}(u))$ (идеально).
 - Добавление ребра: $O(1)$ (добавление в список).
- **Рекомендация (слайд 46):**

- Для **плотных** графов ($(|E| \approx V^2)$) лучше использовать **матрицу смежности**.
- Для **разреженных** графов ($(|E| \ll V^2)$) лучше использовать **списки смежности**.

Г. Матрица инцидентности (Incidence Matrix)

- **Описание:** Матрица размера $(|V| \times |E|)$, где строки — вершины, а столбцы — рёбра (слайд 43-45). В ячейке указывается, инцидентна ли вершина ребру (и как именно).
- **Применение:** Крайне редко используется на практике из-за сложности и неэффективности.



Билет №16: Поиск в глубину (DFS). Алгоритм выделения компонент связности.

1. Поиск в глубину (DFS — Depth First Search)

Идея: Алгоритм идет "вглубь" графа, пока не упрется в тупик (вершину, у которой нет непосещенных соседей). Затем он возвращается назад (отступает) и пробует другие, еще не исследованные ветви (слайд 48, 52).

Алгоритм (рекурсивный):

1. Начинаем из стартовой вершины (v), помечаем её как **посещённую**.
2. Для каждого **непосещённого** соседа (u) вершины (v) рекурсивно запускаем $\text{DFS}(u)$ (слайд 50).

Реализация на Python (слайд 53-57):

```
def dfs(graph: dict[int, list[int]], v: int, visited: set[int] = None):
    if visited is None:
        visited = set()
    visited.add(v)
    print(f"Посетили: {v}") # Обработка вершины
    for neighbor in graph[v]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

- **Сложность:** $(O(V + E))$. Каждая вершина посещается один раз, и каждое ребро просматривается один раз (из списка смежности своей начальной вершины).

2. Поиск компонент связности

Определение:

- **Связный граф:** Граф, в котором между любыми двумя вершинами существует путь.
- **Компонента связности: Максимальный** связный подграф, который нельзя расширить, добавив к нему другие вершины без потери свойства связности (слайд 58-59).

Алгоритм поиска всех компонент связности в неориентированном графе (слайд 60, 64-68):

Поскольку DFS из любой вершины посещает все вершины той компоненты связности, которой эта вершина принадлежит, мы можем найти все компоненты, итерируясь по всем вершинам графа.

1. Инициализируем пустое множество `visited` и пустой список `components`.
2. Для каждой вершины v в графе:
 - Если v ещё не посещена, значит, мы нашли новую компоненту связности.
 - Создаем новый список `component`.
 - Запускаем модифицированный DFS (например, `dfs_collect`), который будет добавлять все посещаемые им вершины в `component`.
 - Добавляем `component` в список `components`.
3. Возвращаем `components`.



Билет №17: DFS в ориентированных графах: проверка на ацикличность и топологическая сортировка (алгоритм на основе DFS и алгоритм Кана).

1. Топологическая сортировка (Topological Sorting)

Задача: Упорядочить вершины **ориентированного ациклического графа (DAG)** в такую линейную последовательность, чтобы для любого ребра $(u \rightarrow v)$ вершина (u) находилась в этой последовательности **левее (раньше)** вершины (v) (слайд 69).

- **Примеры:** Порядок прохождения курсов в университете (сначала математика, потом физика), порядок сборки деталей, зависимости в `package.json`.

Важное условие: Граф должен быть **ациклическим (DAG)**. Если в графе есть цикл, топологическая сортировка невозможна.

2. Алгоритм на основе DFS (рекурсивный)

Идея (слайд 70-73): Используем DFS для обхода графа. Вершина добавляется в результат **в момент выхода из неё** (после того, как все её соседи обработаны). Это гарантирует, что все "зависимые" вершины (те, в которые ведут рёбра) будут добавлены в список раньше, чем сама вершина. Итоговый порядок получается обратным порядку завершения обработки вершин.

Алгоритм (слайд 74):

1. Инициализировать множество `visited` и пустой стек `stack`.
2. Для каждой непосещённой вершины `v` запустить `dfs_topo(v)`:
 - Пометить `v` как посещённую.
 - Для каждого соседа `neighbor` вершины `v`:
 - Если `neighbor` не посещён, рекурсивно вызвать `dfs_topo(neighbor)`.
 - **После обработки всех соседей** (в момент выхода) добавить `v` в стек `stack`.
3. **Обнаружение цикла:** Если в процессе DFS мы наткнулись на ребро, ведущее в вершину, которая уже была посещена, но ещё не "закрыта" (т.е. находится в текущем стеке рекурсии), это означает наличие цикла. Для этого нужно отслеживать не только `visited`, но и "вершины в обработке" (например, отдельным множеством).
4. Результат — содержимое стека, прочитанное снизу вверх, или `stack[::-1]`.

3. Алгоритм Кана (Kahn's Algorithm) — итеративный

Этот алгоритм основан на подсчете полустепеней захода (in-degree) вершин (слайд 75-82).

Идея: В DAG всегда есть хотя бы одна вершина с нулевой полустепенью захода (в неё не ведут рёбра). Такие вершины могут быть первыми в порядке топологической сортировки. Удаляя их, мы уменьшаем степени захода их соседей, открывая путь для новых вершин.

Алгоритм:

1. Вычислить `inDegree[v]` для каждой вершины (v) (количество входящих рёбер).
2. Создать очередь `q` и поместить туда все вершины с `inDegree == 0`.
3. Инициализировать счетчик `visited_count = 0` и список `topo_order`.
4. Пока очередь `q` не пуста:
 - Извлечь вершину `u` из очереди.
 - Добавить `u` в `topo_order`.
 - `visited_count += 1`.
 - Для каждого соседа `v` вершины `u`:

- Уменьшить `inDegree[v]` на 1 (как бы "удаляя" ребро `u -> v`).
- Если `inDegree[v]` стал равен 0, добавить `v` в очередь.

5. После обработки всех вершин:

- Если `visited_count == V`, то граф ацикличен, и `topo_order` содержит корректную топологическую сортировку.
- Если `visited_count < V`, в графе есть цикл, и сортировка невозможна.



Билет №18: Поиск в ширину (BFS). Нахождение кратчайших путей в невзвешенных графах.

1. Поиск в ширину (BFS — Breadth First Search)

Идея: В отличие от DFS, который идет "вглубь", BFS распространяется по графу "**в ширину**", подобно волне от брошенного в воду камня (слайд 2). Алгоритм гарантирует, что вершины посещаются в порядке увеличения расстояния от стартовой вершины (слайд 5).

Алгоритм (слайды 3-5):

1. Начинаем со стартовой вершины `s`, помещаем её в очередь (Queue).
2. Помечаем `s` как посещённую.
3. **Пока очередь не пуста:**
 - Извлекаем вершину `v` из начала очереди.
 - Для каждого **непосещённого** соседа `u` вершины `v`:
 - Помечаем `u` как посещённую.
 - Кладём `u` в конец очереди.

Реализация на Python (слайды 7-13):

```
from collections import deque

def bfs(graph, start_node):
    visited = {start_node}
    queue = deque([start_node])

    while queue:
        v = queue.popleft()
        print(f"Посетили: {v}") # Обработка вершины
        for neighbor in graph[v]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

- **Сложность: $O(V + E)$** , где V — число вершин, E — число рёбер. Каждая вершина попадает в очередь один раз, и каждое ребро просматривается дважды (в неориентированном графе) (слайд 7).

2. Нахождение кратчайших путей в невзвешенном графе

BFS идеально подходит для поиска кратчайшего пути по количеству рёбер, так как он гарантирует, что мы впервые встречаем вершину именно на кратчайшем расстоянии от старта (слайд 14).

Модификация для поиска пути (слайды 15-23):

Чтобы не просто найти вершину, но и восстановить путь к ней, нам понадобятся два вспомогательных массива/словаря:

- `dist`: хранит расстояние (количество рёбер) от старта до каждой посещённой вершины.
- `parent`: хранит предка (вершину, из которой мы пришли) для каждой вершины. Это позволит восстановить весь путь.

Алгоритм `bfs_shortest_path`:

1. Инициализируем `dist[start] = 0`, `parent[start] = None`. Для остальных вершин расстояние можно считать бесконечностью (или просто не записывать).
2. Выполняем стандартный BFS.
3. При первом обнаружении вершины `v` из текущей вершины `u` (`v not in dist`):
 - `dist[v] = dist[u] + 1`
 - `parent[v] = u`
4. Когда мы извлекли из очереди целевую вершину `target`, можно остановиться.
5. **Восстановление пути:** Идем от `target` по ссылкам `parent` до `start`, собирая вершины, а затем разворачиваем полученный список.

```
path = []
u = target
while u is not None:
    path.append(u)
    u = parent[u]
return path[::-1], dist[target]
```



Билет №19: Кратчайшие пути в графах с положительными весами: алгоритм Дейкстры. Особенности реализации с приоритетной очередью.

1. Задача и ограничения

Алгоритм Дейкстры (Dijkstra's algorithm) решает задачу поиска кратчайших путей от одной заданной вершины (источка) до всех остальных вершин в графе. **Важнейшее ограничение:** веса всех рёбер должны быть **неотрицательными** (слайд 9, 22).

2. Идея и алгоритм (слайд 9)

Алгоритм использует жадный подход:

1. **Инициализация:** Расстояние до стартовой вершины `s` устанавливается равным 0 (`d[s] = 0`), а до всех остальных — бесконечность (`d[v] = ∞`). Все вершины считаются непосещёнными.
2. **Основной цикл:** Пока есть непосещённые вершины:
 - Выбрать непосещённую вершину `u` с **минимальным** текущим расстоянием `d[u]`.
 - Пометить `u` как посещённую (кратчайший путь до неё найден и больше не изменится).
 - **Релаксация:** Для каждого соседа `v` вершины `u` проверить, не улучшит ли путь через `u` известное расстояние до `v`. Если `d[v] > d[u] + w(u, v)`, то обновить `d[v] = d[u] + w(u, v)`.

3. Реализация с приоритетной очередью (слайды 11-18)

Для эффективного выбора вершины с минимальным расстоянием на каждом шаге используется **приоритетная очередь (мин-куча)**, например, `heapq` в Python.

Псевдокод:

```
import heapq

def dijkstra(graph, start):
    dist = {node: float('inf') for node in graph}
    dist[start] = 0
    # Очередь хранит пары (расстояние, вершина)
    pq = [(0, start)]

    while pq:
        current_d, u = heapq.heappop(pq)
```



```
# Важная оптимизация: если извлеченное расстояние больше уже известного,
# значит, эта запись устарела (мы нашли более короткий путь позже).
# Пропускаем её.
if current_d > dist[u]:
    continue

for v, weight in graph[u].items():
    if dist[u] + weight < dist[v]:
        dist[v] = dist[u] + weight
        heapq.heappush(pq, (dist[v], v))

return dist
```

Ключевые особенности реализации:

- **Приоритетная очередь:** Позволяет извлекать вершину с наименьшим расстоянием за $O(\log V)$.
- **Проверка устаревших записей:** В очередь могут попасть несколько записей для одной вершины с разными расстояниями (по мере их улучшения). Проверка `if current_d > dist[u]` гарантирует, что мы обрабатываем вершину только с актуальным кратчайшим расстоянием. Без этой проверки алгоритм будет работать, но медленнее.
- **Сложность:** $O(E \log V)$, где E — число рёбер, V — число вершин. Это достигается благодаря приоритетной очереди.

4. Пример работы (слайд 10)

В примере на слайде показана пошаговая работа алгоритма от вершины S до F . Кратчайший путь $S \rightarrow F = 14$, а не $S \rightarrow C \rightarrow B \rightarrow F = 17$.



Билет №20: Кратчайшие пути в DAG (через топологическую сортировку) и графах с отрицательными рёбрами (алгоритм Беллмана-Форда).

1. Кратчайшие пути в DAG (ориентированном ациклическом графе)

Для DAG существует более эффективный алгоритм, чем Дейкстра, работающий за линейное время $O(V + E)$ (слайд 2, 8).

Идея (слайд 2):

Используется рекуррентное соотношение для расстояния

$$d[v] : [d[v] = \min_{(u,v) \in E} (d[u] + w(u, v))]$$

Если обрабатывать вершины в порядке **топологической сортировки**, то к моменту вычисления `d[v]` все значения `d[u]` для входящих в v рёбер уже будут окончательными.

Алгоритм (слайд 8):

1. Выполнить топологическую сортировку графа, получить порядок `order`.
2. Инициализировать `dist[start] = 0`, для остальных `dist[v] = ∞`.
3. Для каждой вершины u в порядке топологической сортировки:
 - Для каждого соседа v вершины u выполнить релаксацию: `dist[v] = min(dist[v], dist[u] + w(u, v))`.
4. Вернуть `dist`.

Преимущество: Алгоритм очень прост и работает за линейное время, что быстрее алгоритма Дейкстры для этого класса графов.

2. Алгоритм Беллмана-Форда (Bellman-Ford Algorithm)

Этот алгоритм решает ту же задачу, что и Дейкстра (поиск кратчайших путей от одного истока), но **может работать с рёбрами отрицательного веса** (слайд 22). Более того, он умеет **обнаруживать отрицательные циклы**, достижимые из истока.

Идея (слайд 23-25): Алгоритм основан на многократной ($|V|-1$ раз) релаксации **всех** рёбер графа. Поскольку простой кратчайший путь (без циклов) не может содержать больше, чем $|V|-1$ ребро, то после $|V|-1$ итераций мы гарантированно найдём все кратчайшие пути.

Алгоритм (слайд 23, 35):

1. **Инициализация:** $\text{dist}[s] = 0$, $\text{dist}[v] = \infty$ для всех остальных v .
2. **Основной цикл ($|V| - 1$ раз):**
 - Для каждого ребра (u, v) с весом w :
 - Если $\text{dist}[u]$ не бесконечность и $\text{dist}[v] > \text{dist}[u] + w$, то выполнить релаксацию: $\text{dist}[v] = \text{dist}[u] + w$.
3. **Проверка на отрицательный цикл:**
 - Выполнить ещё одну ($|V|$ -ю) итерацию по всем рёбрам.
 - Если хотя бы одно расстояние снова уменьшилось, значит, в графе есть **отрицательный цикл**, достижимый из истока, и задача не имеет решения (кратчайший путь не определён).

Сложность: $O(V \cdot E)$ (слайд 26).

Пример работы (слайды 27-34): На слайдах подробно показано, как расстояния постепенно улучшаются на каждой итерации, даже при наличии отрицательного ребра.



Билет №21: Кратчайшие пути между всеми парами вершин: алгоритм Флойда-Уоршелла.

1. Задача

Алгоритм Флойда-Уоршелла (Floyd-Warshall Algorithm) находит кратчайшие пути **между всеми парами вершин** в одном графе (слайд 22, 39).

2. Идея: Динамическое программирование

Ключевая идея — рассматривать промежуточные вершины, через которые может проходить путь (слайд 39).

- **Состояние:** Пусть $\text{dp}[k][i][j]$ — длина кратчайшего пути от i до j , который в качестве **промежуточных** может использовать только вершины из множества $\{1, 2, \dots, k\}$.
- **База:** $\text{dp}[0][i][j]$ — это вес ребра из i в j (если ребра нет, то ∞), и 0 для $i == j$.
- **Переход:** Чтобы найти $\text{dp}[k][i][j]$, есть два варианта:
 1. **Не использовать вершину k :** путь остаётся таким же, как и на предыдущем шаге: $\text{dp}[k-1][i][j]$.
 2. **Использовать вершину k :** тогда путь разбивается на две части: от i до k и от k до j . Их длины равны $\text{dp}[k-1][i][k]$ и $\text{dp}[k-1][k][j]$. Сумма даёт длину пути через k .
- **Рекуррентная формула:**
$$\text{dp}[k][i][j] = \min(\text{dp}[k-1][i][j], \text{dp}[k-1][i][k] + \text{dp}[k-1][k][j])$$

3. Оптимизация по памяти

Поскольку для вычисления $\text{dp}[k]$ нам нужны только значения $\text{dp}[k-1]$, мы можем хранить матрицу "на месте", обновляя её в цикле по k . Это не нарушает корректности, так как на каждой итерации k мы используем уже обновлённые (для текущего k) или старые значения, но логика остаётся верной.

Реализация (слайд 45):

```
def floyd_warshall(matrix, V):
    dist = [row[:] for row in matrix] # Копия матрицы смежности

    for k in range(V):
        for i in range(V):
```

```

        for j in range(V):
            if dist[i][k] != float('inf') and dist[k][j] != float('inf'):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    return dist

```

Сложность: $O(V^3)$ по времени и $O(V^2)$ по памяти.

Пример работы (слайды 40-44): На слайдах показано, как матрица расстояний изменяется по мере увеличения k (разрешённых промежуточных вершин).



Билет №22: Алгоритм A*: использование эвристик для оптимизации поиска пути.

1. Мотивация

Алгоритм Дейкстры находит кратчайший путь, но он "слепой" — он равномерно исследует пространство во всех направлениях, пока не найдёт цель. Это неэффективно для больших графов, особенно если цель известна. Алгоритм A* (A-star) решает эту проблему, используя **эвристику** (интуитивную оценку) для направления поиска в сторону цели (слайд 19).

2. Функция стоимости

В A* используется комбинированная функция стоимости для каждой вершины n :

$$f(n) = g(n) + h(n)$$

- $g(n)$ — фактическая стоимость пути от стартовой вершины до вершины n (как в алгоритме Дейкстры).
- $h(n)$ — **эвристическая оценка** стоимости кратчайшего пути от вершины n до **цели**. Это ключевое отличие от Дейкстры, где $h(n) = 0$ для всех n .

3. Требования к эвристике

Для того чтобы A* гарантированно находил кратчайший путь, эвристика должна быть **допустимой (admissible)**:

- Она никогда не должна **переоценивать** истинную стоимость до цели. То есть $h(n)$ должна быть меньше или равна реальной стоимости пути от n до цели. Это гарантирует, что алгоритм не отбросит потенциально оптимальные пути.

4. Популярные эвристики (для графов на сетке) (слайд 19)

- Манхэттенское расстояние:** Используется, когда разрешено движение только по 4 направлениям (вверх, вниз, влево, вправо).

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

- Евклидово расстояние:** Используется, когда движение возможно в любом направлении (по прямой).

$$h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

5. Алгоритм и связь с Дейкстрой

A* очень похож на алгоритм Дейкстры, но использует приоритетную очередь, где приоритет вершины определяется значением $f(n) = g(n) + h(n)$, а не просто $g(n)$.

- Если $h(n) = 0$ для всех вершин, то $f(n) = g(n)$, и A* полностью эквивалентен алгоритму Дейкстры.
- Благодаря эвристике, A* исследует в первую очередь те вершины, которые, согласно оценке, находятся ближе к цели, что значительно ускоряет поиск.



Билет №23: Минимальные остовные деревья (MST): алгоритм Прима.

1. Остовное дерево и MST (слайд 49)

- **Остовное дерево (Spanning Tree):** Связный подграф, который включает все вершины исходного графа и не содержит циклов (т.е. является деревом). Число рёбер в остовном дереве = $|V| - 1$.
- **Минимальное остовное дерево (MST):** Остовное дерево с **минимальной суммой весов** своих рёбер.

2. Алгоритм Прима (Prim)

Идея (слайд 58-59): Жадный алгоритм, который "растит" дерево из одной стартовой вершины, каждый раз добавляя ребро **минимального веса**, соединяющее построенную часть с новой вершиной.

Алгоритм:

1. Выбрать произвольную стартовую вершину.
2. Использовать приоритетную очередь (мин-кучу) для хранения рёбер, ведущих из построенной части к непосещённым вершинам.
3. Пока очередь не пуста:
 - Извлечь ребро с минимальным весом.
 - Если вершина уже посещена, пропустить.
 - Добавить вершину в MST, обновить общий вес.
 - Добавить в очередь все рёбра из новой вершины к непосещённым вершинам.

Реализация (слайд 62-65):

```
import heapq
def prim(n, adj):
    visited = [False] * n
    pq = [(0, 0)] # (weight, vertex)
    mst_weight = 0
    while pq:
        w, u = heapq.heappop(pq)
        if visited[u]:
            continue
        visited[u] = True
        mst_weight += w
        for v, weight in adj[u]:
            if not visited[v]:
                heapq.heappush(pq, (weight, v))
    return mst_weight
```

Сложность: $O(E \log V)$. Хорошо работает на **плотных графах**.



Билет №24: Бинарные деревья: основные свойства и типы обходов.

1. Основные свойства деревьев (слайд 2)

Определение дерева: Дерево — это **связный ациклический граф**.

Свойства:

- Число рёбер в дереве на единицу меньше числа вершин: $|E| = |V| - 1$.
- Между любыми двумя вершинами существует **ровно один** путь.

Ориентированное дерево:

- Ровно одна вершина имеет нулевую степень захода — **корень**.
- Все остальные вершины имеют степень захода 1.
- Вершины с нулевой степенью исхода называются **листьями**.

2. Бинарное дерево (слайд 4)

Это дерево, в котором каждый узел может иметь не более двух потомков — **левого** и **правого**.

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.data = key
```

3. Типы обходов бинарного дерева (слайд 6-7)

Обход	Порядок	Применение	Результат для дерева на слайде 7
Прямой (Pre-order, NLR)	Корень → Лево → Право	Копирование дерева	1-2-4-5-3-6-7
Центрированный (In-order, LNR)	Лево → Корень → Право	Для BST — выдаёт отсортированный порядок	4-2-5-1-6-3-7
Обратный (Post-order, LRN)	Лево → Право → Корень	Удаление дерева, вычисление выражений	4-5-2-6-7-3-1
Уровневый (Level-order, BFS)	Послойно, сверху вниз, слева направо	Использует очередь	1-2-3-4-5-6-7



Билет №25: Бинарные деревья поиска (BST): операции поиска, вставки и удаления узла. Анализ высоты дерева.

1. Определение BST (слайд 8)

Бинарное дерево поиска (Binary Search Tree, BST) обладает свойством упорядоченности:

Для любого узла X:

- Все ключи в **левом поддереве** меньше ключа X.
- Все ключи в **правом поддереве** больше ключа X.

2. Операции

А. Поиск (Search) (слайд 9):

```
def search(node, key):
    if not node or node.data == key:
        return node
    if key < node.data:
        return search(node.left, key)
    return search(node.right, key)
```

Б. Вставка (Insert) (слайд 18):

```
def insert(node, key):
    if node is None:
        return Node(key)
    if key < node.data:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)
    return node
```

В. Удаление (Delete) (слайд 19-21):

- 1. Нет детей (лист): Просто удаляем узел.
- 2. Один ребёнок: Заменяем удаляемый узел его единственным потомком.
- 3. Два ребёнка: Находим **следующий по величине** узел (самый левый в правом поддереве), копируем его значение в удаляемый узел, а затем рекурсивно удаляем этот successor.

Г. Поиск следующего элемента (Successor) (слайд 10-17):

- 1. Если у узла есть правое поддерево, successor — минимальный элемент в этом поддереве.
- 2. Если правого поддерева нет, нужно подняться вверх, пока узел не станет левым потомком своего родителя. Тогда родитель — successor.

3. Анализ высоты и сложности (слайд 8, 22)

Случай	Высота	Сложность операций
Сбалансированное дерево	$\approx \log_2 n$	$O(\log n)$
Вырожденное дерево ("бамбук")	$= n$	$O(n)$

Проблема (слайд 22): При вставке ключей в отсортированном порядке (1, 2, 3, 4, 5) дерево вырождается в связный список. **Решение:** Самобалансирующиеся деревья (AVL, красно-чёрные).



Билет №26: Сбалансированные деревья поиска: обзор AVL-деревьев и концепции вращений.

1. Зачем нужны сбалансированные деревья?

Обычное BST в худшем случае (при вставке отсортированных данных) может выродиться в связный список, и сложность операций станет **$O(n)$** . Сбалансированные деревья гарантируют, что высота дерева всегда остаётся **$O(\log n)$** .

2. AVL-деревья (слайд 32-37)

Определение: AVL-дерево — это сбалансированное двоичное дерево поиска, в котором для каждой вершины высоты левого и правого поддеревьев различаются **не более чем на 1** (слайд 32).

Баланс-фактор: Для каждого узла хранится `diff = h(left) - h(right)`. Допустимые значения: -1, 0, 1. Если после вставки или удаления `diff` становится 2 или -2, требуется **балансировка** (слайд 33).

3. Концепция вращений (Rotations)

Для восстановления баланса используются **повороты** — локальные перестроения поддеревьев (слайд 33):

Тип вращения	Условие	Описание
Малое правое (Right Rotation)	Left-Left (LL)	Левый потомок становится корнем, исходный корень — его правым потомком
Малое левое (Left Rotation)	Right-Right (RR)	Правый потомок становится корнем, исходный корень — его левым потомком
Большое правое (Left-Right Rotation)	Left-Right (LR)	Сначала малое левое вращение вокруг левого потомка, затем малое правое вокруг текущего узла
Большое левое (Right-Left Rotation)	Right-Left (RL)	Сначала малое правое вращение вокруг правого потомка, затем малое левое вокруг текущего узла

4. Алгоритм вставки в AVL-дерево (слайд 34)

- 1. Выполнить вставку как в обычном BST.
- 2. Поднимаясь вверх по пути к корню, пересчитать баланс-факторы.

3. Если в какой-то вершине $diff = 2$ или $diff = -2$, выполнить соответствующее вращение.

4. Сложность: $O(\log n)$.

5. AVL vs Красно-чёрные деревья (слайд 38)

Характеристика	AVL-дерево	Красно-чёрное дерево
Баланс	Строгий (разница высот ≤ 1)	Менее строгий (цветовые свойства)
Поиск	Быстрее (меньше высота)	Немного медленнее
Вставка/Удаление	Медленнее (больше вращений)	Быстрее (меньше вращений)
Применение	Когда поиск преобладает над изменениями	Универсальное (STL, Java)



Билет №27: Жадные алгоритмы: принцип жадного выбора и оптимальная подструктура. Задача о выборе интервалов и непрерывный рюкзак.

1. Что такое жадные алгоритмы (слайд 4-10)

Жадный алгоритм — это алгоритм, который на каждом шаге выбирает **локально оптимальное решение** (лучшее прямо сейчас), не думая о последствиях для будущих шагов (слайд 10).

Если на каждом шаге мы выбираем «лучшее, что можем взять сейчас», и в итоге получаем глобально оптимальный результат — задача решается жадно.

Пример интуиции: Выдача сдачи монетами номиналом 25, 10, 5, 1 копейка. Стратегия: брать самую крупную монету, которая помещается. Для канонической системы номиналов это даёт оптимальное минимальное количество монет.

Где применяются (слайд 4-9):

- Планирование задач (задача о выборе интервалов)
- Сжатие данных (алгоритм Хаффмана)
- Маршрутизация (алгоритм Дейкстры)
- Оптимизация затрат (размен монет, непрерывный рюкзак)
- Сетевые алгоритмы (построение минимального остовного дерева — Прим, Крускал)

2. Условия применимости (слайд 11-14)

Жадный подход корректен, когда задача обладает двумя свойствами:

1. **Свойство жадного выбора (Greedy Choice Property):** Локально оптимальный выбор входит в некоторый глобально оптимальный ответ. Мы можем принять решение сейчас, без учёта будущих шагов (слайд 12, 15).
2. **Оптимальная подструктура (Optimal Substructure):** После жадного выбора остаётся подзадача того же типа, чей оптимальный ответ, вместе с текущим выбором, даёт оптимальный ответ исходной задачи (слайд 13, 23-26).

Важно: Жадные алгоритмы работают не для всех задач. Например, для дискретного рюкзака жадность не даёт оптимальное решение (слайд 14, 36-39).

3. Задача о выборе интервалов (Interval Scheduling) (слайд 40-55)

Условие: Дан набор интервалов $[s_i, f_i)$ (время начала и конца). Найти максимальное число **непересекающихся** интервалов (слайд 40).

Жадная стратегия (слайд 40):

1. Отсортировать интервалы по **времени окончания** f_i (по возрастанию).
2. На каждом шаге выбирать интервал с наименьшим f_i , который не пересекается с ранее выбранным (начинается после $last_end$).

Реализация (слайд 51-55):

```
def max_non_overlapping(intervals):
    intervals.sort(key=lambda x: x[1]) # Сортировка по времени окончания
    count = 0
    last_end = -1
    for s, f in intervals:
        if s >= last_end:
            count += 1
            last_end = f
    return count
```

Сложность: $O(N \log N)$ — определяется сортировкой. Память: $O(1)$.

Доказательство корректности (обменный аргумент) (слайд 44-50):

1. Пусть i_1 — интервал с наименьшим f_{i_1} (первый в жадном выборе).
2. Пусть $O = \{o_1, o_2, \dots, o_k\}$ — оптимальное решение, упорядоченное по f .
3. Если $i_1 \neq o_1$, заменяем o_1 на i_1 . Поскольку $f_{i_1} \leq f_{o_1}$, все последующие интервалы в O начинаются после f_{o_1} , значит, и после f_{i_1} . Набор остаётся допустимым, размер не уменьшился.
4. Индукция по шагам завершает доказательство.

4. Непрерывная задача о рюкзаке (Fractional Knapsack) (слайд 56-67)

Условие: Есть n предметов, у каждого вес w_i и стоимость v_i . Рюкзак вместимости W . Предметы можно **делить на части** (можно взять, например, половину предмета). Максимизировать стоимость содержимого (слайд 56).

Жадная стратегия (слайд 56):

1. Отсортировать предметы по **удельной стоимости** v_i / w_i (стоимость на единицу веса) в порядке **убывания**.
2. Брать предметы в этом порядке, пока есть место. Последний предмет (который целиком не помещается) берём **частично**.

Пример (слайд 58-61): $w = [10, 20, 30]$, $v = [60, 100, 120]$, $W = 50$.

- Удельные стоимости: 6, 5, 4.
- Берём предмет 1 (весь, 60, осталось 40).
- Берём предмет 2 (весь, 100, осталось 20).
- Берём предмет 3 частично: 20 из 30 → стоимость $120 * 20/30 = 80$.
- Итого: $60 + 100 + 80 = 240$.

Почему работает (слайд 57): Функция стоимости **линейна** по весу. Нет «бонуса» за взятие целого предмета. Всегда выгоднее взять грамм более дорогого предмета.

Реализация (слайд 62-67):

```
def fractional_knapsack(weights, values, capacity):
    items = sorted(range(len(weights)),
                    key=lambda i: values[i] / weights[i], reverse=True)
    total_value = 0.0
    remaining = capacity
    for i in items:
        take = min(weights[i], remaining)
        total_value += values[i] * (take / weights[i])
        remaining -= take
    return total_value
```



Билет №28: Оптимальное префиксное кодирование: алгоритм Хаффмана.

1. Постановка задачи (слайд 68)

Передать текст по каналу связи с минимальным числом бит.

- **Фиксированный код:** каждый символ кодируется k битами. Для алфавита из n символов: $k = \lceil \log_2 n \rceil$.
- **Переменный код:** частые символы получают короткий код, редкие — длинный.
- **Префиксное свойство:** код ни одного символа не является префиксом кода другого. Это гарантирует **однозначность декодирования**.

2. Идея алгоритма Хаффмана (слайд 69-73)

Строится **дерево Хаффмана** — бинарное дерево:

- **Листья** — символы с их частотами.
- **Внутренние узлы** — сумма частот детей.
- **Код символа** = путь от корня к листу (0 — влево, 1 — вправо).

Жадный шаг (слайд 73): На каждом шаге берём **два узла с наименьшими частотами** и объединяем их в новый внутренний узел. Повторяем, пока не останется один корень.

3. Пример построения (слайд 74-80)

Частоты: [5, 9, 12, 13, 16, 45]

1. Берём 5 и 9 → узел 14.
2. Берём 12 и 13 → узел 25.
3. Берём 14 и 16 → узел 30.
4. Берём 25 и 30 → узел 55.
5. Берём 45 и 55 → корень 100.

Коды (пример):

- 45 → 0
- 16 → 101
- 13 → 110
- 12 → 111
- 9 → 1000
- 5 → 1001

4. Реализация (слайд 81-88)

```
import heapq
from collections import deque

def huffman_codes(freqs):
    heap = [(f, ch) for ch, f in freqs.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        f1, node1 = heapq.heappop(heap)
        f2, node2 = heapq.heappop(heap)
        heapq.heappush(heap, (f1 + f2, (node1, node2)))
```

```

root = heap[0][1]
codes = {}
queue = deque([(root, '')])
while queue:
    node, prefix = queue.popleft()
    if isinstance(node, str):
        codes[node] = prefix
    else:
        queue.append((node[0], prefix + '0'))
        queue.append((node[1], prefix + '1'))
return codes

```

Сложность: $O(N \log N)$ по времени, $O(N)$ по памяти.

5. Доказательство оптимальности (слайд 89-97)

Лемма: В оптимальном префиксном коде два наименее частых символа — листья-братья на максимальной глубине.

Доказательство от противного (обменный аргумент):

- Пусть x, y — наименее частые, но не братья.
- Пусть a, b — братья на максимальной глубине. Тогда $f_x \leq f_a$ и $f_y \leq f_b$.
- При обмене позиций $x \leftrightarrow a, y \leftrightarrow b$ длина кодов для x, y увеличивается, для a, b — уменьшается.
- $\Delta = (f_a - f_x) + (f_b - f_y) \geq 0$ — обмен не уменьшает общий вес кода.
- Значит, исходное расположение было не хуже. Противоречие.

Следствие: Первый шаг Хаффмана (объединение двух наименее частых) совпадает с оптимальным решением. Индукция завершает доказательство.



Билет №29: Принципы динамического программирования: мемоизация (Top-down) и табуляция (Bottom-up).

1. Что такое динамическое программирование (слайд 2)

Динамическое программирование (Dynamic Programming, DP) — способ решения сложных задач путём разбиения их на более простые **перекрывающиеся подзадачи**. Запоминание результатов решения подзадач позволяет избежать повторных вычислений.

2. Ключевые свойства (слайд 3-4)

- **Оптимальная подструктура:** Оптимальное решение задачи можно построить из оптимальных решений её подзадач (слайд 3).
- **Перекрывающиеся подзадачи:** Одна и та же подзадача возникает многократно при решении основной задачи (слайд 4).

3. Мемоизация (Memoization) — Top-down подход (слайд 5, 19)

Идея: Рекурсивное решение "сверху вниз" с кэшированием результатов.

Алгоритм:

1. Написать рекурсивную функцию для решения задачи.
2. Создать структуру данных (обычно массив или словарь) для хранения уже вычисленных результатов.
3. В начале рекурсивного вызова проверить: если результат уже есть в кэше, вернуть его.
4. Иначе вычислить результат рекурсивно и сохранить в кэше.

Преимущества: Вычисляются только нужные подзадачи; код ближе к математической рекуррентной формуле.

Недостатки: Риск переполнения стека при большой глубине рекурсии.

4. Табуляция (Tabulation) — Bottom-up подход (слайд 5, 20-21)

Идея: Итеративное решение "снизу вверх", начиная с самых маленьких подзадач.

Алгоритм (слайд 6):

- 1. Свести задачу для N к задаче для чисел, меньших N.
- 2. Создать массив (таблицу) для хранения результатов.
- 3. Заполнить начальные (базовые) значения (для которых формула не работает).
- 4. Обойти массив в цикле и заполнить остальные значения по формуле.
- 5. Вернуть ответ из массива.

Преимущества: Нет рекурсии; можно оптимизировать использование памяти (хранить не все значения, а только необходимые).

Недостатки: Вычисляются все подзадачи, даже если они не нужны для ответа.

5. Сравнение подходов

Характеристика	Мемоизация (Top-down)	Табуляция (Bottom-up)
Направление	От большой задачи к маленьким	От маленьких задач к большой
Реализация	Рекурсивная + кэш	Итеративная (циклы)
Вычисление	Только нужных подзадач	Всех подзадач (обычно)
Риск	Переполнение стека	Нет
Оптимизация памяти	Сложнее	Проще (можно хранить только необходимое)



Билет №30: Классические задачи DP: числа Фибоначчи, дискретная задача о рюкзаке, наибольшая общая подпоследовательность (LCS).

1. Числа Фибоначчи (слайд 7-22)

Определение: $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ при $n \geq 2$ (слайд 7).

Наивная рекурсия (слайд 10):

```
def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n-2)
```

Сложность: $O(2^n)$ — экспоненциально (слайд 9).

Мемоизация (слайд 19):

```
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
```

Сложность: $O(n)$ по времени, $O(n)$ по памяти.

Табуляция (слайд 20):

```
def fib_tabulation(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

Сложность: $O(n)$ по времени, $O(n)$ по памяти.

Оптимизированная табуляция (слайд 21):

```
def fib_optimized(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for i in range(2, n + 1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current
    return prev1
```

Сложность: $O(n)$ по времени, **$O(1)$** по памяти.

2. Дискретная задача о рюкзаке (0/1 Knapsack) (слайд 29-37)

Условие (слайд 29): Есть n предметов, у каждого вес w_i и стоимость p_i . Рюкзак вместимости w . Каждый предмет можно взять **только один раз** (0 или 1). Максимизировать стоимость.

DP состояние (слайд 30): $S[i][j]$ — максимальная стоимость, которую можно получить, используя первые i предметов и имея вместимость j .

Рекуррентное соотношение (слайд 30):

$$S[i][j] = \max(S[i-1][j], p_i + S[i-1][j - w_i])$$

- $S[i-1][j]$ — не берём i -й предмет.
- $p_i + S[i-1][j - w_i]$ — берём i -й предмет (если $j \geq w_i$).

База: $S[0][j] = 0$ для всех j (нет предметов — стоимость 0).

Пример (слайд 31-37): $W = 13$, предметы: $(w_1=3, p_1=1)$, $(w_2=4, p_2=6)$, $(w_3=5, p_3=4)$, $(w_4=8, p_4=7)$, $(w_5=9, p_5=6)$.

Реализация:

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, W + 1):
            if j >= weights[i-1]:
                dp[i][j] = max(dp[i-1][j],
                               values[i-1] + dp[i-1][j - weights[i-1]])
            else:
                dp[i][j] = dp[i-1][j]
    return dp[n][W]
```

Сложность: $O(N \cdot W)$ по времени и памяти. Можно оптимизировать память до $O(W)$, используя одномерный массив (итерируя j от W вниз).

3. Наибольшая общая подпоследовательность (LCS) (слайд 23-28)

Определение (слайд 23): Для двух последовательностей $\{a_i\}$ и $\{b_j\}$ найти **самую длинную** подпоследовательность (не обязательно непрерывную), которая является общей для обеих.

Пример (слайд 24-26): $a = [1, 2, 3, 4, 5]$, $b = [2, 7, 3, 2, 5]$. $LCS = [2, 3, 5]$, длина = 3.

DP состояние: $LCS(p, q)$ — длина НОП для префиксов $a[1..p]$ и $b[1..q]$.

Рекуррентное соотношение (слайд 27):

```
[
LCS(p, q) = \begin{cases}
LCS(p-1, q-1) + 1, & \text{если } a_p = b_q \\
\max(LCS(p, q-1), LCS(p-1, q)), & \text{если } a_p \neq b_q
\end{cases}
]
```

Реализация (слайд 28):

```
def lcs(a, b):
    n, m = len(a), len(b)
    table = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i-1] == b[j-1]:
                table[i][j] = table[i-1][j-1] + 1
            else:
                table[i][j] = max(table[i-1][j], table[i][j-1])
    return table[-1][-1]
```

Сложность: $O(n \cdot m)$ по времени и памяти. Память можно оптимизировать до $O(\min(n, m))$, храня только текущую и предыдущую строки.